LangGraph's Position in the Stack

LangGraph focuses **only on agent orchestration**

It does **not** abstract:

- Models
- Prompts
- Tools

- Gives developers **fine-grained control** over agent execution.
- This is why LangGraph feels "low-level" compared to other frameworks.

How LangGraph and LangChain Are Related

- LangGraph **can be used independently**
- LangGraph **can also work together with LangChain**
  - LangChain is often used to provide models and tools
- Both solve **different problems** in AI agent development
  - **LangChain** focuses on *building agent logic*
  - **LangGraph** focuses on *running and controlling agents*

LangGraph – Installation & Hello World Example

Installation

- LangGraph can be installed using **pip** or **uv**:

```
pip install -U langgraph
```

Sample Program

```python
from langgraph.graph import StateGraph, MessagesState, START, END

def mock_llm(state: MessagesState):
    return {"messages": [{"role": "ai", "content": "hello world"}]}

graph = StateGraph(MessagesState)
graph.add_node(mock_llm)
graph.add_edge(START, "mock_llm")
graph.add_edge("mock_llm", END)

graph = graph.compile()

print(graph.invoke({
    "messages": [{"role": "user", "content": "hi!"}]
}))
```

LangGraph – Installation & Hello World Example

Output

```
{'messages': [HumanMessage(content='hi!', additional_kwargs={}, response_metadata={}, id='89bcd704-80c1-489d-a67b-16a440e7ec6a'), AIMessage(content='hello world', additional_kwargs={}, response_metadata={}, id='9075b0ed-7a2d-46e0-9860-98d8c94ac705', tool_calls=[], invalid_tool_calls=[])]}
PS C:\Users\NikilEdwinEdwinDavid\Desktop\training\Agentic AI\LangGraph>
```

Code Explanation

```python
from langgraph.graph import StateGraph, MessagesState, START, END
```

What these are:

- **StateGraph** – Core class used to define a graph-based workflow
- **MessagesState** – Predefined state schema for chat-style messages
- **START** – Special node representing the entry point of the graph
- **END** – Special node representing the exit point of the graph

LangGraph – Installation & Hello World Example

```
def mock_llm(state: MessagesState):
    return {"messages": [{"role": "ai", "content": "hello world"}]}
```

What happens here:

- This function represents a **graph node**
- It receives the current **state** (messages so far)
- Returns an updated state with a new AI message

```
graph = StateGraph(MessagesState)
```

What happens here:

- Initializes a graph with a defined state structure
- Ensures all nodes read and write data in the same format

LangGraph – Installation & Hello World Example

```
graph.add_node(mock_llm)
```

What happens here:

- Registers `mock_llm` as a node in the graph
- Node name defaults to the function name (`mock_llm`)

```
graph.add_edge(START, "mock_llm")
graph.add_edge("mock_llm", END)
```

What happens here:

- Defines execution order
- Flow: **START → mock_llm → END**

LangGraph – Installation & Hello World Example

```
graph = graph.compile()
```

What happens here:

- Validates graph structure
- Converts it into an executable workflow

```
graph.invoke({
    "messages": [{"role": "user", "content": "hi!"}]
})
```

What happens here:

- Starts execution from Start
- Passes initial state into the graph
- Returns the final state after reaching END

Difference Between Langchain & LangGraph

Dimension	LangChain	LangGraph
Agent loop	Framework-managed ReAct loop — you don't control iteration	You define the loop — full control over when to iterate, stop, or branch
State	Lost after each run — no memory of past executions, unless explicity maintained	Persistent state — survives across steps
Flow control	Agent decides tool calls, but you can't intervene mid-loop	You control every transition — add conditions, approvals, retries at any node
Looping	Implicit — agent loops until it decides it's done	Explicit — you define when and why the agent loops back
Human-in-the-loop	Not built-in — hard to pause and resume	Native — pause at any node, wait for input, resume from where it stopped
Multi-agent	Hacky — chaining agents together is manual wiring	Agents as nodes in a graph, hand off state between them naturally
Error recovery	Agent fails → restart from scratch	Fail at node 4 → retry from node 4 with full state intact

Difference Between Normal AI Agent & LangGraph

LangChain (Agent):

A reactive agent — gets a query, reasons, picks a tool, observes the result, and repeats until it has an answer. The agent loop (Reason → Act → Observe) is managed by LangChain internally. You define the tools and the prompt, LangChain handles the loop. Works great when the agent can solve the task in a single reasoning cycle without needing memory of past runs or complex branching.

LangGraph (Graph-based Agent):

A controlled, stateful agent — you explicitly define the nodes (steps) and edges (transitions) as a graph. The agent moves through the graph based on conditions, can loop back, branch, wait for human input, or persist state across runs.

Scenario:

An agent that handles customer refund requests. Simple cases ("what's my refund status?" → tool call → answer) work fine as a LangChain agent. But when the agent needs to verify identity → check policy → maybe escalate to human → wait for approval → loop back to ask the customer for more info → then process — that's a graph. The flow isn't a single ReAct cycle, it branches and loops based on what happens at each step. That's LangGraph.

When to Use Langchain vs LangGraph

When to go with LangChain Agent:

- Single agent, single task — "answer thisquestion using these tools"
- ReAct pattern is enough — reason, act,observe, done
- No need to pause, persist,

When to go with LangGraph Agent:

- You don't know the agent flow upfront — it needs to decidedynamically
- Agent needs to loop with self-correction ("that answerwasn't good enough, try again")
- Multi-agent handoff — one agent's output feeds intoanother based on conditions
- Long-running tasks — agent pauses, state persists, resumeshours later
- Error recovery — retry from the failed step, not from thebeginning

Core Capabilities of LangGraph

- **Durable execution** (survives failures)
- **Streaming** (real-time outputs and state updates)
- Human-in-the-loop workflows
- Stateful and long-running agents